SRS — Biblioteca Inteligente

Nov 4, 2026 · @Mirllan

Información del documento

Este SRS define Biblioteca Inteligente, un catálogo web con votos en tiempo real y recomendaciones por IA, construido sobre la aplicación Django que ya funciona.

Campo	Valor
Curso	Electiva V
Entrega	Entrega 2 (punto 14 de la guía): núcleo funcional
Versión	2.0
Integrantes	David Humberto Mahecha Cardozo y Edilson Mirllan Suarez Gutiérrez
Repositorio	github.com/David78-beto/Biblioteca-Inteligente
Video Entrega 1	youtube.com/watch?v=1tX8ytETkqM
Video Entrega 2	Por completar cuando se publique
Aplicación publicada	biblioteca-inteligente-glj7.onrender.com

1. Introducción

El documento sirve al equipo de desarrollo y al docente como acuerdo de qué se construye y con qué arquitectura. Su estructura se inspira en la norma ISO/IEC/IEEE 29148, adaptada al alcance de un proyecto académico.

Término	Definición
SRS	Especificación de requisitos de software
RF / RNF	Requisito funcional / requisito no funcional
Embedding	Vector numérico que representa el significado de un texto

Término	Definición
WebSocket	Canal bidireccional que permite al servidor enviar datos al navegador
Contenedor	Paquete aislado con una aplicación y sus dependencias, ejecutado con Docker

2. Problema a resolver

Los lectores de una biblioteca pequeña no tienen una forma sencilla de saber qué libros hay ni cuál leer a continuación.

- **Quién lo tiene:** lectores y bibliotecarios de bibliotecas escolares o comunitarias sin un sistema de gestión propio.
- **Cómo se ve actualmente:** el catálogo suele vivir en listas o en hojas de cálculo. Encontrar un libro exige preguntar o revisar los estantes, y las recomendaciones dependen de la memoria del bibliotecario.
- **Por qué vale la pena resolverlo:** un catálogo consultable reduce el trabajo repetitivo del bibliotecario, y las recomendaciones acercan a cada lector a libros que quizá nunca habría encontrado.

Pregunta abierta: este planteamiento es un supuesto del equipo. Conviene validarlo con al menos una biblioteca real antes de la Entrega 2.

3. Justificación

El proyecto se justifica porque resuelve una necesidad concreta con herramientas de código abierto y permite aplicar cada tecnología del curso a un problema distinto.

- **Valor para el usuario:** une en un solo lugar la consulta del catálogo, la opinión de otros lectores y las recomendaciones.
- **Costo bajo:** todo el software elegido es gratuito, y la API de IA generativa se usará con un límite de consumo controlado.
- **Riesgo controlado:** el catálogo en Django ya funciona, de modo que las entregas siguientes agregan capacidades sobre una base probada.
- **Valor académico:** el equipo practica una arquitectura de servicios con Django, Node.js, IA, Docker y una base de datos relacional.

4. Objetivos

Objetivo general: desarrollar una aplicación web contenerizada que permita consultar y gestionar el catálogo de una biblioteca, y que recomiende libros mediante inteligencia artificial.

Objetivos específicos:

1. Gestionar géneros y libros con operaciones de consulta, creación, edición y eliminación en Django.
2. Registrar el interés de los lectores mediante votos y mostrarlos en tiempo real con un servicio en Node.js.
3. Recomendar libros similares y permitir búsquedas en lenguaje natural con un servicio de IA.
4. Empaquetar los servicios con Docker, levantarlos con Docker Compose y publicar la aplicación en la nube.
5. Mantener la calidad del código con pruebas automatizadas e integración continua en GitHub.

5. Alcance

El proyecto cubre el catálogo, los votos, las notificaciones y las recomendaciones de una sola biblioteca; no cubre los préstamos físicos.

Incluido:

- Catálogo de géneros y libros, con administración.
- Registro e inicio de sesión de usuarios, con un voto por libro y usuario.
- Votos y notificaciones en tiempo real.
- Recomendaciones por similitud y búsqueda en lenguaje natural.
- Despliegue público en Render con Docker, y Docker Compose para el entorno local.

Excluido:

- Préstamos, devoluciones y multas.
- Pagos y compra de libros.
- Lectura de libros digitales.
- Aplicación móvil nativa.

Supuestos: el catálogo se escribe en español y cabe en un solo servidor.

Estado actual (Entrega 2): Django ya tiene el CRUD del catálogo, el registro, el inicio de sesión, los roles Lector, Bibliotecario y Administrador, un voto por usuario, el panel de administración y 32 pruebas automatizadas. La aplicación se empaqueta en un contenedor Docker y se publica en Render con PostgreSQL. El microservicio Node.js, la API REST, la integración con IA generativa y Swagger quedan para la Entrega 3.

6. Tecnologías

Cada tecnología tiene una responsabilidad única, y solo Django escribe en la base de datos del catálogo.

Tecnología	Responsabilidad	Motivo de la elección	Estado
Django 6.1 (Python 3.14)	Interfaz web, autenticación, roles, administración y lógica del catálogo	Incluye ORM, panel de administración y autenticación	En uso
Gunicorn y WhiteNoise	Servir Django y sus archivos estáticos en producción	Servidor estándar de Python; los estáticos no necesitan otro servidor	En uso
PostgreSQL	Base de datos de producción	Relacional y gestionada por Render	En uso
SQLite	Base de datos de desarrollo	No requiere instalación	En uso
Docker	Empaquetar Django en un contenedor	El mismo entorno en desarrollo y en Render	En uso
Render	Publicar la aplicación y la base de datos	Despliega desde GitHub e incluye HTTPS y un plan gratuito	En uso
Git, GitHub y GitHub Actions	Versiones, ramas `main` y `develop`, y pruebas en cada envío	Exigido por la guía; deja evidencia de cada cambio	En uso

Tecnología	Responsabilidad	Motivo de la elección	Estado
Docker Compose	Levantar todos los servicios juntos en local	Un solo comando para el entorno completo	Parcial: hoy Django y PostgreSQL
Node.js con Express y Socket.IO	Microservicio de recomendaciones y eventos en tiempo real	Exigido por la guía; maneja muchas conexiones abiertas	Entrega 3
Swagger/OpenAPI	Documentación pública de la API del microservicio	Exigido por la guía	Entrega 3
API de IA generativa	Recomendaciones y búsqueda en lenguaje natural	Exigida por la guía; proveedor por definir	Entrega 3

7. Requisitos funcionales

Doce de los diecisiete requisitos ya están implementados en Django; los otros cinco se planifican para las entregas siguientes.

ID	El sistema debe…	Actor	Prioridad	Estado
RF-01	Listar los géneros disponibles	Lector	Alta	Implementado
RF-02	Listar los libros de un género	Lector	Alta	Implementado
RF-03	Mostrar el detalle de un libro	Lector	Alta	Implementado
RF-04	Permitir agregar un libro	Bibliotecario	Alta	Implementado
RF-05	Permitir editar un libro	Bibliotecario	Alta	Implementado
RF-06	Eliminar un libro tras pedir confirmación	Bibliotecario	Alta	Implementado

ID	El sistema debe…	Actor	Prioridad	Estado
RF-07	Registrar un voto en un libro	Lector	Media	Implementado
RF-08	Administrar géneros y libros desde un panel	Administrador	Media	Implementado
RF-09	Permitir el registro y el inicio de sesión	Lector	Alta	Implementado
RF-10	Limitar a un voto por usuario y libro	Lector	Alta	Implementado
RF-11	Restringir la gestión del catálogo al personal autorizado	Bibliotecario	Alta	Implementado
RF-12	Exponer el catálogo mediante una API REST	Microservicio Node.js	Alta	Planificado
RF-13	Actualizar los votos en pantalla sin recargar	Lector	Media	Planificado
RF-14	Notificar en pantalla la llegada de libros nuevos	Lector	Baja	Planificado
RF-15	Recomendar libros similares en la página de detalle	Lector	Alta	Planificado
RF-16	Buscar libros con una frase en lenguaje natural	Lector	Media	Planificado
RF-17	Guardar una descripción de cada libro, que el servicio de IA usa para recomendar y buscar	Bibliotecario	Alta	Implementado

8. Requisitos no funcionales

Los requisitos siguientes tienen una condición que se puede comprobar; las metas de tiempo son objetivos del equipo y aún no se han medido.

ID	Atributo	Requisito	Cómo se verifica
RNF-01	Rendimiento	Las páginas del catálogo responden en menos de 2 s en local	Medición con las herramientas del navegador
RNF-02	Tiempo real	Un voto se ve en otro navegador en menos de 1 s	Prueba manual con dos navegadores
RNF-03	Disponibilidad	Si el servicio de IA falla, el catálogo sigue funcionando	Prueba con el contenedor de IA detenido
RNF-04	Seguridad	Las claves se leen de variables de entorno y nunca se versionan	Revisión del repositorio
RNF-05	Seguridad	Toda acción que cambia datos usa POST con protección CSRF	Pruebas automatizadas
RNF-06	Portabilidad	El sistema se levanta con `docker compose up`	Ejecución en un equipo limpio
RNF-07	Mantenibilidad	Las pruebas se ejecutan en cada envío a GitHub	Registro de GitHub Actions
RNF-08	Usabilidad	La interfaz está en español y se adapta a pantallas de celular	Revisión a 375 px de ancho
RNF-09	Seguridad	Las contraseñas cumplen las reglas de Django y se guardan cifradas	Pruebas automatizadas de registro
RNF-10	Seguridad	En producción todo el tráfico usa HTTPS y las cookies de sesión son seguras	Revisión de la URL publicada
RNF-11	Autorización	Cada acción comprueba el rol; sin permiso, responde con error 403	Pruebas automatizadas de roles

9. Diagramas UML

Los diagramas muestran lo construido hasta la Entrega 2 y, con línea punteada, lo planificado para la Entrega 3.

Casos de uso

Seis casos de uso ya funcionan y tres llegan en la Entrega 3

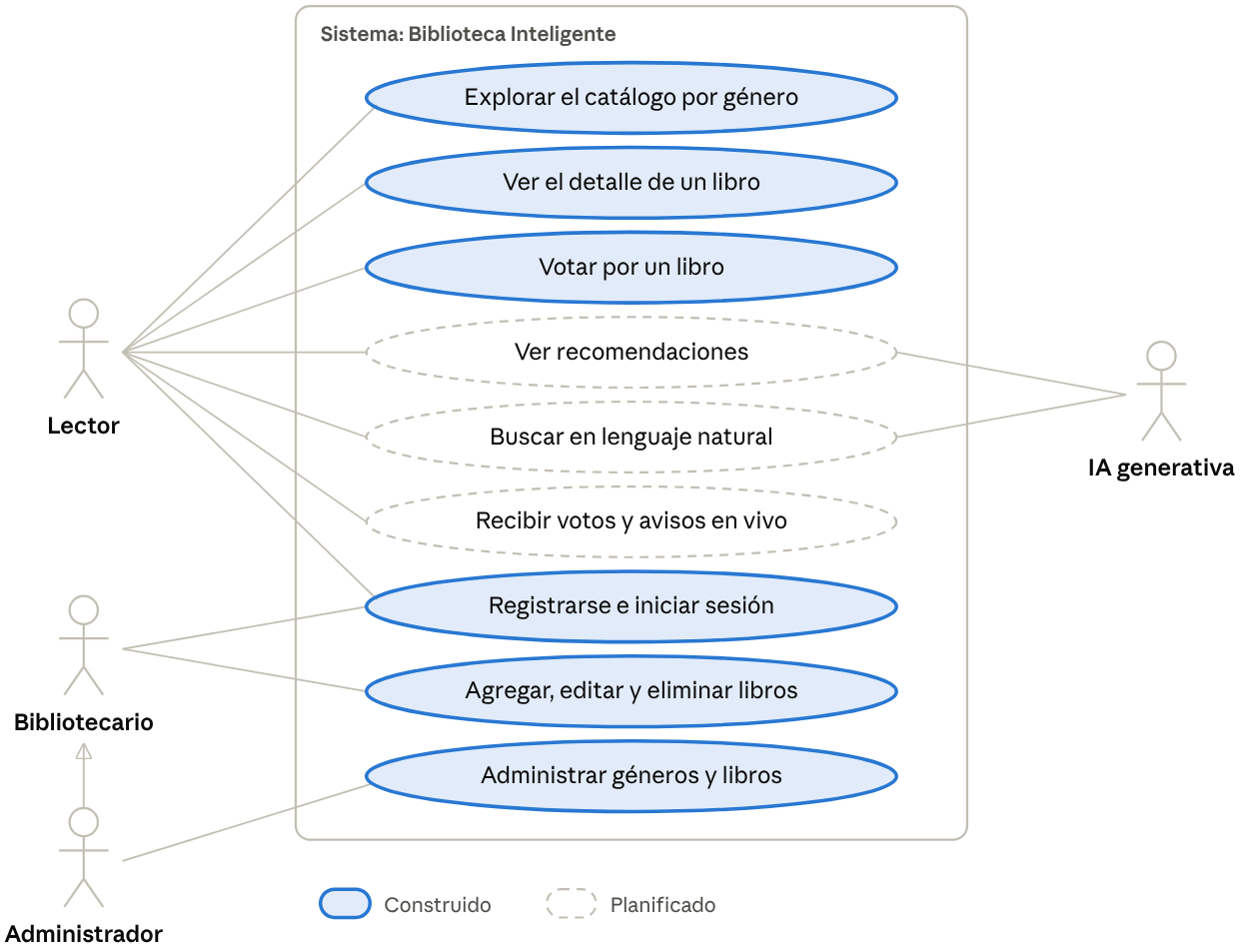


Diagrama de casos de uso · 9 casos, 4 actores

El Administrador hereda los casos del Bibliotecario. El Servicio de IA es un actor secundario: responde a las recomendaciones y a las búsquedas.

Componentes

Django ya funciona con usuarios y roles; Node.js y la IA llegan en la Entrega 3

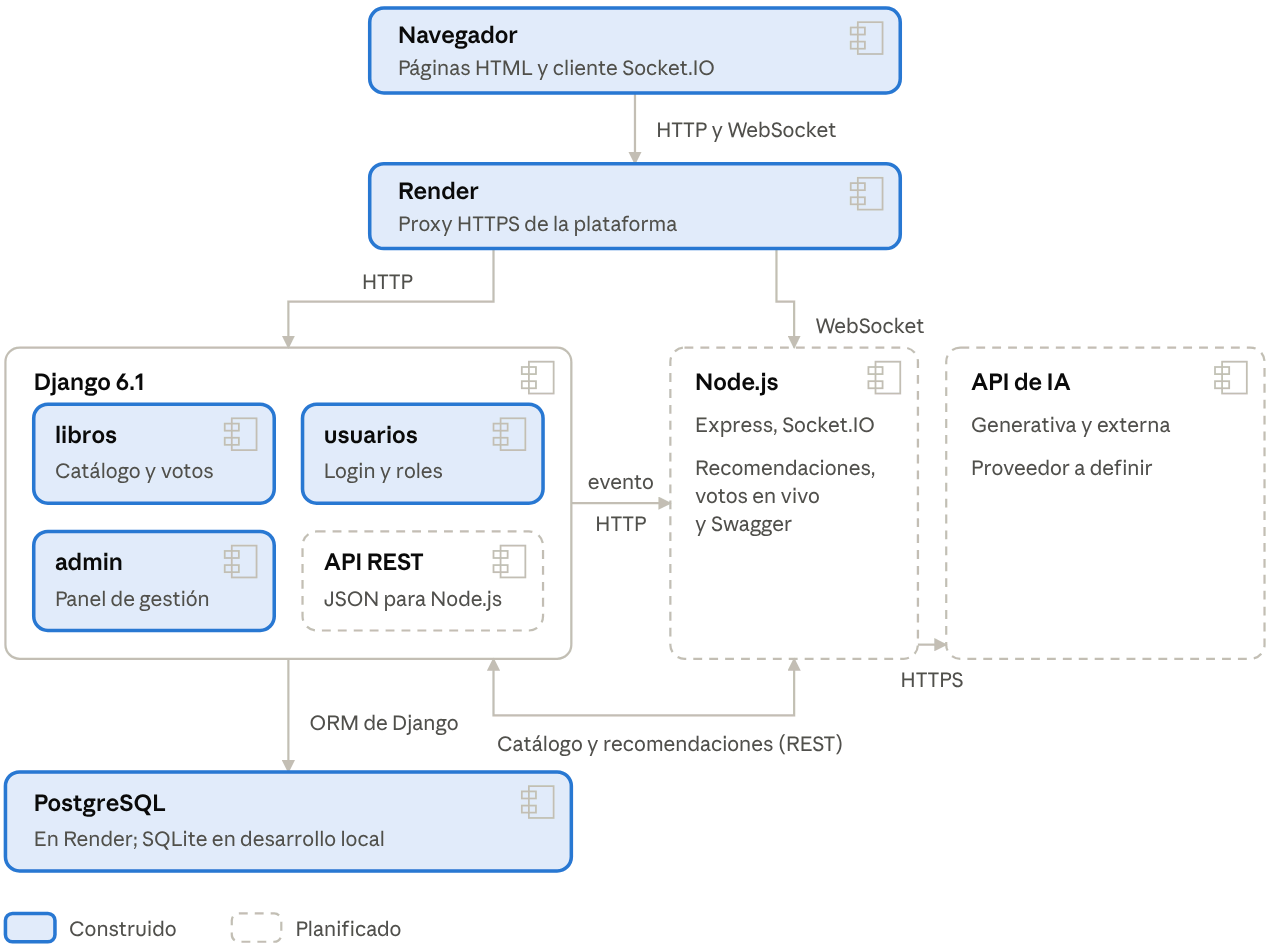


Diagrama de componentes · Django, Node.js, IA y base de datos

Render recibe el tráfico por HTTPS y lo entrega a Django, que guarda los datos en PostgreSQL mediante el ORM. En la Entrega 3, Django avisará de cada voto al microservicio Node.js, y este pedirá las recomendaciones a la API de IA generativa.

Despliegue

Render publica Django en Docker con PostgreSQL; Node.js se suma en la Entrega 3

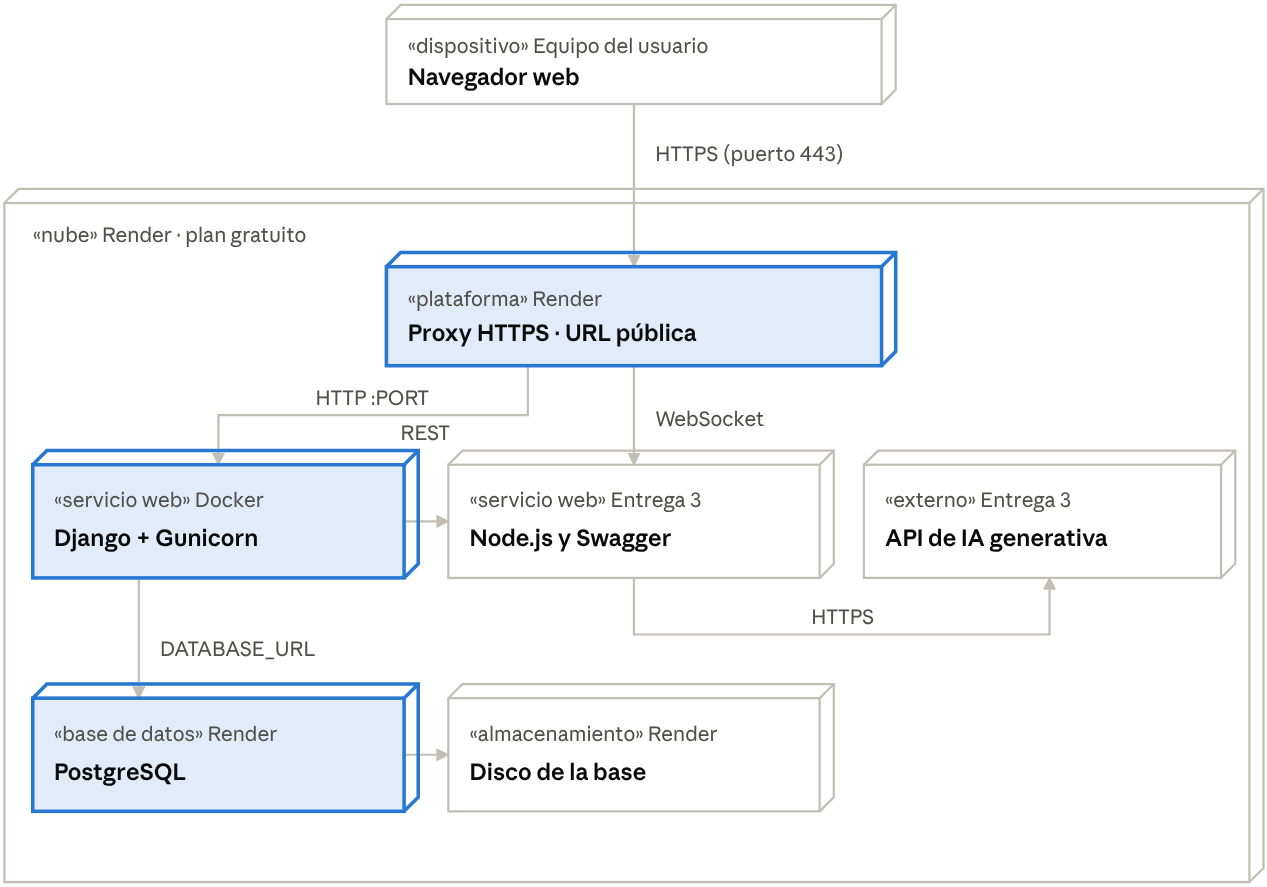


Diagrama de despliegue · 5 contenedores y 2 volúmenes

Cada envío a la rama `main` de GitHub hace que Render construya la imagen de Docker y publique la nueva versión. Al arrancar, el contenedor aplica las migraciones, carga el catálogo inicial si la base está vacía y crea el administrador. En local, Docker Compose levanta Django y PostgreSQL con un solo comando.

Secuencia: registro

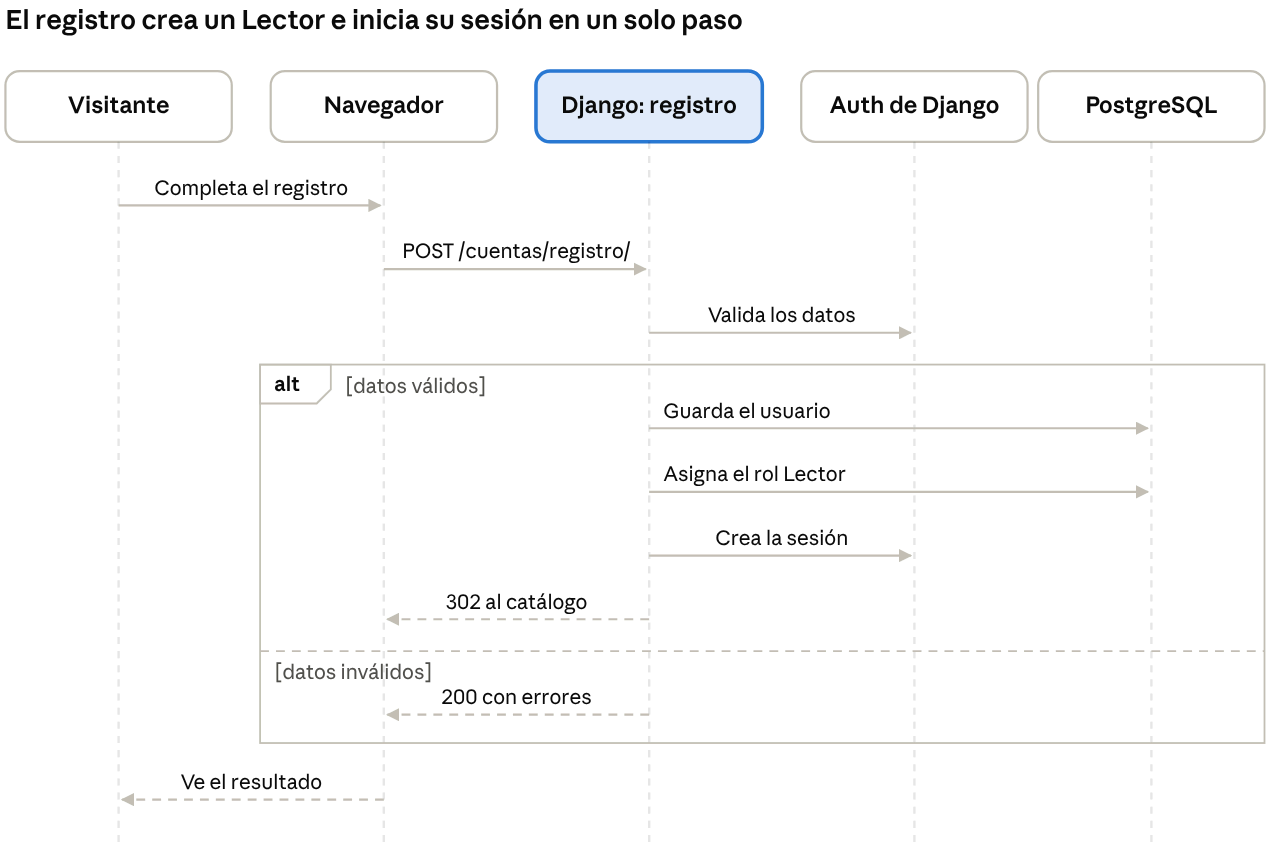


Diagrama de secuencia · registro e inicio de sesión

La contraseña se valida y se guarda cifrada antes de crear la sesión. Si los datos no son válidos, el formulario vuelve con los errores y no se crea ningún usuario.

Secuencia: crear un libro

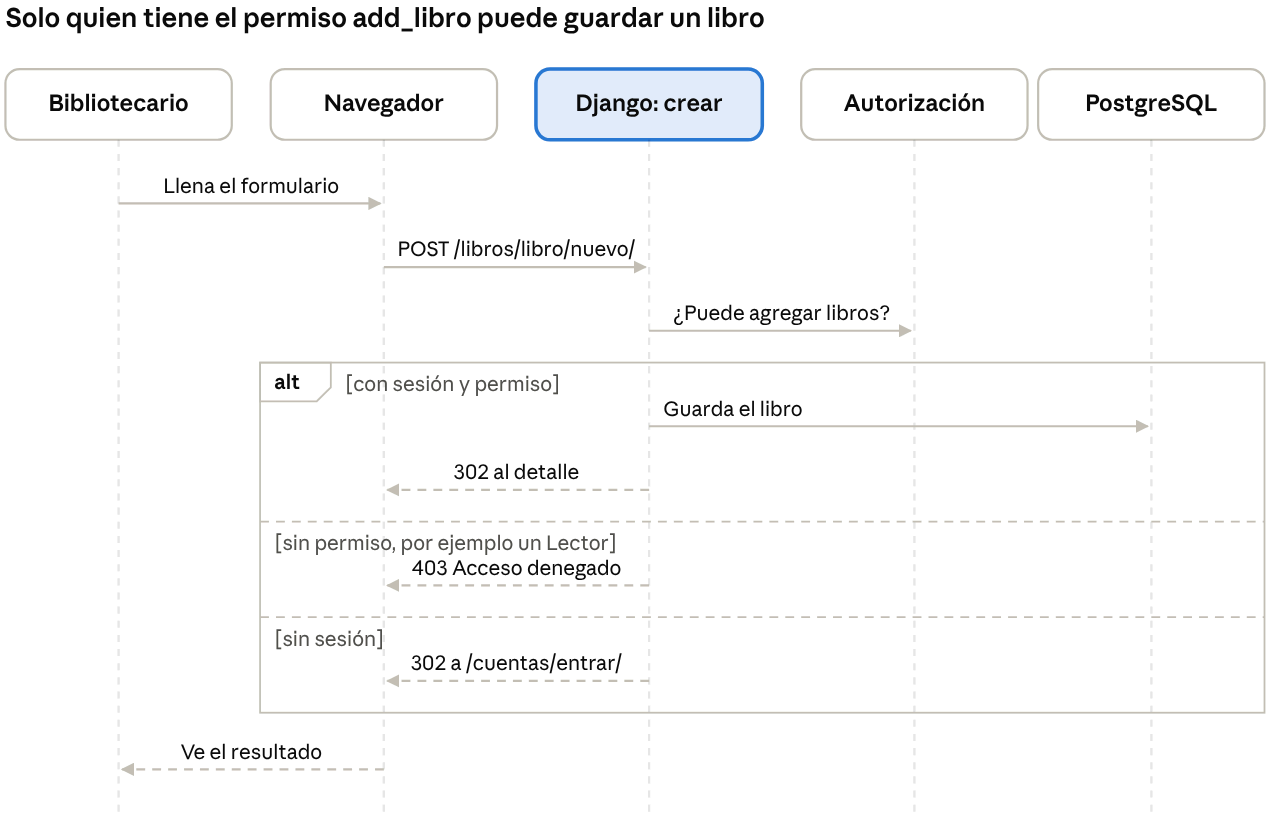


Diagrama de secuencia · crear un libro con control de rol

Django comprueba primero que haya una sesión y después el permiso del rol. Editar y eliminar siguen el mismo camino con los permisos `change_libro` y `delete_libro`.

10. Backlog inicial

El backlog tiene 13 historias ordenadas por prioridad: 8 ya están hechas y 5 quedan para las entregas siguientes.

ID	Historia de usuario	Criterio de aceptación	RF	Prioridad	Estado
HU-01	Como lector, quiero explorar los libros por género para encontrar qué leer	Al elegir un género veo solo sus libros; un género inexistente muestra error 404	RF-01, RF-02, RF-03	Alta	Hecho

ID	Historia de usuario	Criterio de aceptación	RF	Prioridad	Estado
HU-02	Como bibliotecario, quiero agregar, editar y eliminar libros para mantener el catálogo al día	Los cambios se guardan, se confirma cada acción y eliminar pide confirmación	RF-04, RF-05, RF-06	Alta	Hecho
HU-03	Como lector, quiero votar por un libro para mostrar que me gustó	El contador sube en 1 con cada voto enviado por POST	RF-07	Media	Hecho
HU-04	Como administrador, quiero un panel para gestionar géneros y libros	Puedo buscar y filtrar libros y crear géneros en `/admin/`	RF-08	Media	Hecho
HU-05	Como lector, quiero crear una cuenta para tener una identidad en el sistema	Puedo registrarme, entrar y salir; una contraseña incorrecta se rechaza	RF-09	Alta	Hecho
HU-06	Como lector, quiero que mi voto cuente una sola vez para que el ranking sea justo	Un segundo voto del mismo usuario no cambia el contador	RF-10	Alta	Hecho
HU-07	Como bibliotecario, quiero que solo el personal modifique el catálogo para proteger los datos	Un lector sin permiso recibe error 403 al intentar editar	RF-11	Alta	Hecho
HU-08	Como microservicio Node.js, quiero leer el catálogo por API para pedir recomendaciones a la IA	`GET /api/libros/` devuelve los libros en JSON, con título, autor, género y descripción	RF-12	Alta	Por hacer
HU-13	Como bibliotecario, quiero registrar la descripción de cada libro para que la IA pueda recomendarlo y encontrarlo	El formulario guarda la descripción y el detalle la muestra; un libro sin descripción sigue siendo válido	RF-17	Alta	Hecho
HU-09	Como lector, quiero recibir recomendaciones similares para descubrir más libros	El detalle muestra hasta 5 libros, sin incluir el actual; si la IA falla, la página carga sin ellos	RF-15	Alta	Por hacer
HU-10	Como lector, quiero ver los votos cambiar en vivo para saber qué interesa ahora	Un voto aparece en otro navegador sin recargar la página	RF-13	Media	Por hacer
HU-11	Como lector, quiero buscar con una frase para encontrar un libro sin saber su título	«Historia sobre un planeta desértico» incluye Dune entre los primeros resultados	RF-16	Media	Por hacer

ID	Historia de usuario	Criterio de aceptación	RF	Prioridad	Estado
HU-12	Como lector, quiero saber cuándo llega un libro nuevo para no perdérmelo	Con la página abierta aparece un aviso al agregarse un libro	RF-14	Baja	Por hacer

11. Actores y roles

Cada cuenta nueva queda como Lector; solo el Administrador puede convertirla en Bibliotecario desde el panel de Django.

Actor	Tipo	Qué puede hacer	Cómo obtiene el rol
Visitante	Humano, sin sesión	Consultar géneros, libros y su detalle	No necesita cuenta
Lector	Humano	Lo del Visitante, más votar una vez por libro	Al registrarse (grupo Lector)
Bibliotecario	Humano	Lo del Lector, más agregar, editar y eliminar libros	El Administrador lo asigna al grupo Bibliotecario
Administrador	Humano	Todo, incluido el panel de Django y la gestión de usuarios y géneros	Superusuario creado al desplegar
Microservicio Node.js	Sistema, Entrega 3	Leer el catálogo por API y emitir eventos en vivo	Credenciales de servicio
API de IA generativa	Sistema externo, Entrega 3	Generar recomendaciones y resolver búsquedas	Clave de API en variables de entorno

12. Restricciones

- **Tecnológicas:** la guía exige Django, Node.js, GitHub, Docker, Docker Compose, Swagger/OpenAPI, PostgreSQL, Render y una API de IA generativa.
- **Plataforma:** el plan gratuito de Render suspende la aplicación tras un tiempo sin visitas, así que la primera carga puede tardar. Su base de datos gratuita vence a los 30 días.

- **Seguridad:** las claves y contraseñas solo viven en variables de entorno, nunca en el repositorio.
- **Calendario:** Entrega 2 el 12/11/2026, Entrega 3 el 19/11/2026 y sustentación el 26/11/2026.
- **Equipo:** dos integrantes; el trabajo se integra con las ramas `develop` y `main`.

13. Matriz de trazabilidad

Doce requisitos ya tienen módulo implementado y evidencia en el Video 2; los cinco restantes se cierran en la Entrega 3. Las rutas se suman a la URL publicada de la tabla inicial.

RF	Caso de uso	Historia	Componente	Evidencia en video	Entrega	Ruta publicada
RF-01	Explorar el catálogo por género	HU-01	Vista `index`	Video 2, demo	1	`/libros/`
RF-02	Explorar el catálogo por género	HU-01	Vista `genero`	Video 2, demo	1	`/libros/genero/<id>/`
RF-03	Ver el detalle de un libro	HU-01	Vista `detail`	Video 2, demo	1	`/libros/libro/<id>/`
RF-04	Agregar, editar y eliminar libros	HU-02	Vista `crear` y `LibroForm`	Video 2, CRUD	1	`/libros/libro/nuevo/`
RF-05	Agregar, editar y eliminar libros	HU-02	Vista `editar`	Video 2, CRUD	1	`/libros/libro/<id>/editar/`
RF-06	Agregar, editar y eliminar libros	HU-02	Vista `eliminar`	Video 2, CRUD	1	`/libros/libro/<id>/eliminar/`
RF-07	Votar por un libro	HU-03	Vista `votar`	Video 2, demo	1	`/libros/libro/<id>/votar/`
RF-08	Administrar géneros y libros	HU-04	Django Admin (`libros/admin.py`)	Video 2, roles	1	`/admin/`
RF-09	Registrarse e iniciar sesión	HU-05	Vista `registro`, `LoginView`, `LogoutView`	Video 2, registro y login	2	`/cuentas/registro/` y `/cuentas/entrar/`
RF-10	Votar por un libro	HU-06	Modelo `Voto` con restricción única	Video 2, demo	2	`/libros/libro/<id>/votar/`
RF-11	Agregar, editar y eliminar libros	HU-07	Grupos Lector y Bibliotecario, `permission_required`	Video 2, roles	2	`/libros/libro/nuevo/` (403 para Lector)
RF-12	Ver recomendaciones	HU-08	API REST de Django	Video 3	3	Planificado

RF	Caso de uso	Historia	Componente	Evidencia en video	Entrega	Ruta publicada
RF-13	Recibir votos y avisos en vivo	HU-10	Microservicio Node.js con Socket.IO	Video 3	3	Planificado
RF-14	Recibir votos y avisos en vivo	HU-12	Microservicio Node.js con Socket.IO	Video 3	3	Planificado
RF-15	Ver recomendaciones	HU-09	Node.js y API de IA generativa	Video 3	3	Planificado
RF-16	Buscar en lenguaje natural	HU-11	Node.js y API de IA generativa	Video 3	3	Planificado
RF-17	Agregar, editar y eliminar libros	HU-13	Campo `Libro.descripcion`	Video 2, CRUD	2	`/libros/libro/<id>/`